Checking Whether PySliceKit's "Significant" Flags Actually Hold Up

Anshuman Tiwari
Noida Institute of Engineering and Technology, Greater Noida

Abstract

PySliceKit is a tool that automatically splits a dataset into smaller groups and checks whether a machine learning model performs noticeably worse on any of them than it does overall, flagging a group as a genuine problem only if a statistical test comes back significant. Because the tool checks many groups in a single run, it faces a well-known statistical trap: running many tests at once inflates the chance that some will look significant purely by luck. We checked whether PySliceKit's flagged results hold up once standard correction (Bonferroni and Benjamini-Hochberg) is applied, across four public datasets covering both regression and classification models. Before correction, 14 groups were flagged as significant. After Benjamini-Hochberg correction, only 10 survived, and the pattern was not random: groups tested with a simple statistical test (classification tasks) were almost entirely robust, while groups tested with a resampling-based test (regression tasks) mostly lost their significance. While investigating this, we found and fixed a real bug in the tool's resampling test, which was reporting the wrong kind of number as its p-value. After the fix, we identified a hard resolution limit built into the resampling test that can make it mathematically impossible for even a genuinely strong result to survive correction once enough groups are tested together. Every number reported here was independently recalculated by hand and matched the tool's own output across all four datasets with zero disagreements.

Keywords

model auditing, subgroup analysis, multiple-comparisons correction, false discovery rate, Benjamini-Hochberg, bootstrap resampling, statistical significance, machine learning fairness, software testing

1. Introduction

When a machine learning model is evaluated, the usual practice is to report one overall number – for example, "92% accuracy." This can hide a great deal. A model can perform very well across most of a dataset while quietly failing on a specific subgroup: a particular age range, a particular region, or some combination of features. The overall number never reveals this on its own.

To catch this kind of hidden weakness, a growing number of tools automatically slice a dataset into subgroups, measure a model's performance separately on each one, and flag the subgroups where performance drops noticeably. Because a drop could simply be random noise from a small group, these tools typically run a statistical significance test before flagging anything as a real problem.

This is exactly where a subtle issue can appear. If a tool checks twenty or thirty subgroups in one run, and none of them actually have a real problem, a handful would still be expected to look "significant" purely by chance. This is not a flaw in any individual test; it is a direct consequence of running many tests at once without adjusting for it, and it is a well-documented issue in statistics called the multiple-comparisons problem.

We used PySliceKit, an open-source Python library built for exactly this kind of subgroup auditing, as a case study. We chose it because it is fully open, actively maintained, and clearly structured enough to inspect its statistical logic directly rather than treating it as a black box. Our aim was to answer a simple question: do PySliceKit's flagged subgroups still look significant once we account for the number of tests it ran to find them?

We make three contributions. First, we show across four real datasets that PySliceKit's significance flags are not equally reliable across task types: classification results held up well under correction, while regression results mostly did not. Second, in the process of this audit, we found and fixed an actual bug in the library's regression significance test, which was silently reporting an incorrect statistic. Third, after the fix, we identify and empirically demonstrate a hard resolution limit in the corrected test that can make genuinely strong effects mathematically unable to survive correction once enough subgroups are checked at once, and we confirm a straightforward fix for it.

2. Literature Review

2.1 Slice-based model auditing

The practice of slicing a dataset into subgroups and checking model performance separately on each one builds on earlier work in subgroup discovery and fairness auditing, and now appears in several practical tools, including Fairlearn [12], Aequitas [11], and Google's Slice Finder [10]. These tools generally follow a similar pattern: continuous features are binned into ranges, categorical values are grouped, and sometimes two or more columns are combined to search for effects that only appear jointly (for example, "young and from a specific region" rather than either factor alone). PySliceKit follows this same pattern: it automatically bins numeric columns into quartiles, builds every single-column and paired-column combination up to a fixed depth, and runs a significance test on each resulting subgroup.

We did not systematically audit whether Fairlearn, Aequitas, or similar tools apply multiple-comparisons correction internally, and we make no claim about their behavior. What this

report demonstrates is that the underlying problem exists, concretely and measurably, in at least one real, currently used tool of this kind, and that the mechanism producing it (many simultaneous significance tests, without correction) is general enough that it plausibly applies wherever the same pattern is used elsewhere.

2.2 The multiple-comparisons problem

Running a single hypothesis test at the standard 5% significance threshold carries a 5% chance of a false positive when there is genuinely no effect present. Running twenty independent tests under the same conditions raises the chance that at least one looks falsely significant to roughly 64%. This is not a defect in any individual test; each behaves exactly as intended. It is a direct mathematical consequence of testing many things at once without adjustment.

Two standard corrections address this. **Bonferroni correction** divides the significance threshold by the number of tests performed, guaranteeing that the chance of even a single false positive across all tests stays at or below the chosen level. It is simple and conservative, though it can miss real effects as the number of tests grows large. **Benjamini-Hochberg (BH) correction** [1] instead controls the expected proportion of false positives among everything flagged as significant, which is a softer standard than Bonferroni and tends to preserve more true findings at the cost of allowing a small, controlled share of them to be false. A later extension by Benjamini and Yekutieli [2] addresses the case where tests are not fully independent, which is relevant to PySliceKit's design (see Section 6). Storey's positive false discovery rate [4] offers a further alternative not explored here. Both methods used in this report operate entirely on the raw p-values produced by the underlying tests; if those p-values are computed incorrectly, the correction inherits that error regardless of how carefully it is applied.

2.3 PySliceKit's testing logic

PySliceKit selects one of three statistical tests for each subgroup depending on the task and the sample size. For classification tasks with 30 or more samples in the subgroup, it uses a two-proportion z-test [9] comparing the subgroup's accuracy to the overall dataset accuracy. For classification tasks with fewer than 30 samples, it switches to Fisher's exact test [8], since the z-test's assumptions become unreliable at small sample sizes. For regression tasks, where accuracy has no direct meaning, it uses a bootstrap confidence interval test built on the resampling framework of Efron [6, 7]: it resamples the subgroup's data with replacement 1,000 times, computes the metric on each resample, and checks whether the overall dataset's value falls outside the resulting range. Subgroups below a minimum sample-size threshold are shown in the output but excluded from testing and marked as unreliable.

Dataset	Task	Model	Metric	Columns
Calif. Housing [20]	Regr.	Random Forest [26]	MAE [30]	House age, ocean prox.
Breast Cancer [21, 22]	Class.	Logistic Reg. [27]	F1 [29]	Radius, texture
Adult Income [23]	Class.	Random Forest [26]	Accuracy	Sex, race
Diabetes [24]	Regr.	Ridge Reg. [28]	MAE [30]	Age grp., BMI grp.

Table 1: Datasets and models audited in this study.

2.4 Permutation and resampling p-value discreteness

Statistical tests built by resampling data a fixed number of times, rather than relying on a closed-form formula, are known to have a built-in resolution limit: the smallest possible non-zero p-value they can report is fixed by how many resamples were taken. This has been documented specifically by Phipson and Smyth [3], who caution that permutation and resampling p-values should never be reported or treated as exactly zero because of this same discreteness. Our contribution is not the discovery of this phenomenon in general, which is established in the statistics literature, but a concrete demonstration of how it plays out inside a real, currently used machine learning auditing tool, at its default settings, on real data.

3. Method

3.1 Datasets and models

We audited four public datasets, selected to cover both regression and classification tasks and both test paths used by PySliceKit:

For each dataset, we trained the model in the ordinary way, held out a test set, and ran PySliceKit's `evaluate()` function at its standard cross-product depth of two, meaning it checked both single-column subgroups and every pairwise combination of the two chosen columns. Sample-size thresholds followed each dataset's natural scale.

3.2 Applying correction

For each dataset, we took the full list of subgroup results returned by PySliceKit and applied Bonferroni and Benjamini-Hochberg correction separately, both at the standard 5% level. Only subgroups where a test had actually run were included in the correction; subgroups skipped for having too few samples, or where the underlying metric could not be computed, were excluded, since including them would misrepresent how many genuine tests were performed.

3.3 Independent verification

We did not rely on the correction results on their own. For every dataset, we separately reimplemented both the BH and Bonferroni procedures from scratch, using only the raw p-values and no external statistics libraries, and compared the outcome against the library's own output for every sub-

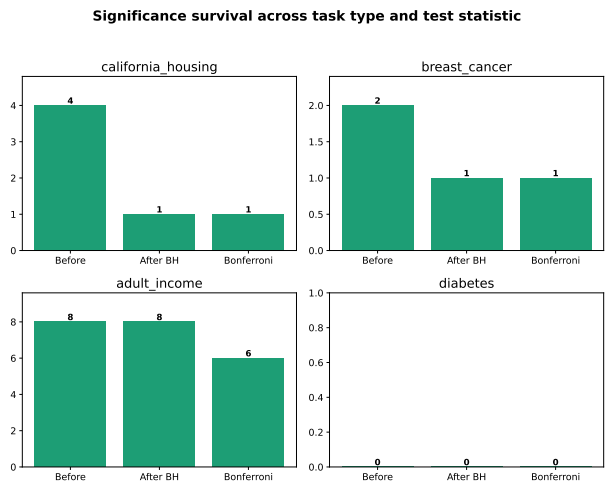


Figure 1: Number of significant subgroups before correction, after Benjamini-Hochberg correction, and after Bonferroni correction, for each of the four datasets.

Dataset	Test	N	Bef.	BH	Bonf.
Calif. Housing	Bootstrap	24	4	1	1
Breast Cancer	z/Fisher	11	2	1	1
Adult Income	z/Fisher	16	8	8	6
Diabetes	Bootstrap	7	0	0	0

Table 2: Correction outcomes by dataset. "Test" refers to the significance test PySliceKit selected: bootstrap confidence interval for regression tasks, or two-proportion z-test / Fisher's exact test for classification tasks.

group individually. Any disagreement would have been investigated; none occurred. All four datasets passed with full agreement between the independent calculation and the library's output. The complete comparison files, listing every subgroup's raw p-value alongside both correction outcomes, are included as supplementary material.

4. Results

Table 2 summarizes how many subgroups were flagged significant before correction, and how many survived each correction method. Column headers are abbreviated below purely for layout; "N" is the number of subgroups tested, "Bef." is significant before correction, "BH" is significant after Benjamini-Hochberg correction, and "Bonf." is significant after Bonferroni correction.

The two classification datasets lost either none or very few of their originally significant subgroups. The two regression datasets lost almost everything, or had nothing significant to begin with; the Diabetes row reflects a null result rather than correction removing findings, since no subgroup crossed the significance threshold even before correction was applied.

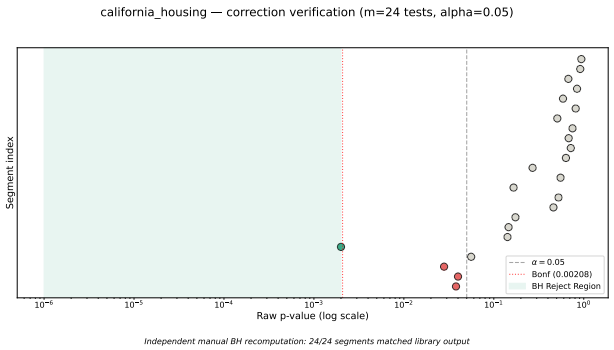


Figure 2: California Housing. Most subgroup results sit well above both the uncorrected 5% cutoff and the stricter Bonferroni threshold. One subgroup survives, and only narrowly.

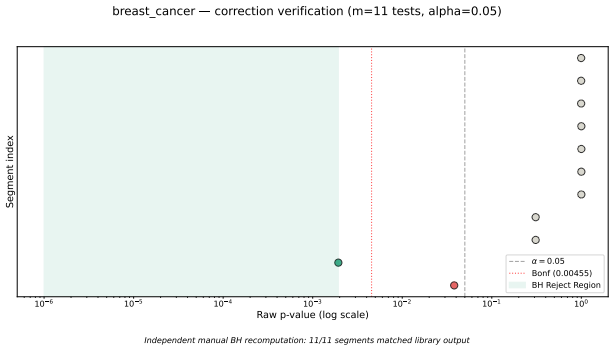


Figure 3: Breast Cancer. One of two originally flagged subgroups survives correction, with a clear gap between the surviving and discarded results.

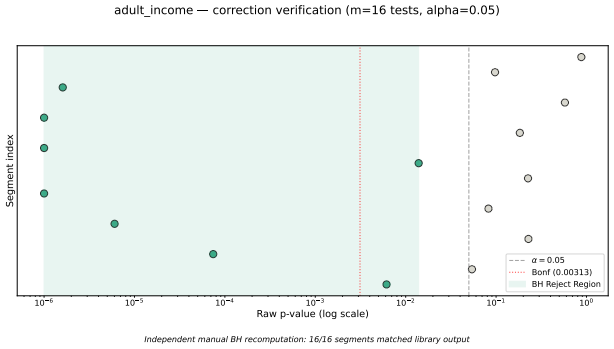


Figure 4: Adult Income. All eight originally flagged subgroups survive Benjamini-Hochberg correction; six of eight survive the stricter Bonferroni correction.

4.1 Why the bootstrap-based test is more fragile

The bootstrap test resamples the data 1,000 times and counts how often the resampled result was at least as extreme as the real one. Because this count can only take a whole-number value out of 1,000, the resulting p-value can only fall on a fixed grid, and the smallest possible non-zero value it can ever report is 2 divided by 1,000, or 0.002.

In the California Housing audit, 24 subgroups were tested at once, producing a Bonferroni threshold of approximately 0.00208 – narrowly above the test's own floor of 0.002. The one subgroup that survived correction did so only because

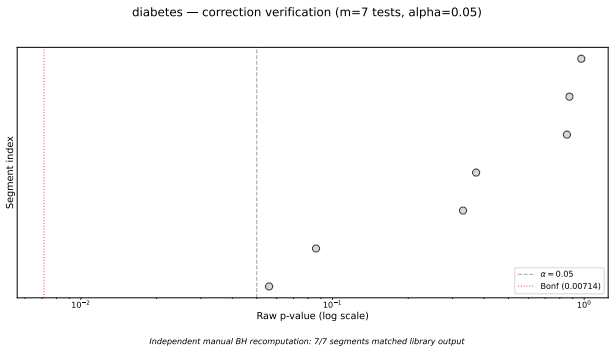


Figure 5: Diabetes. No subgroup crosses the significance threshold at any stage.

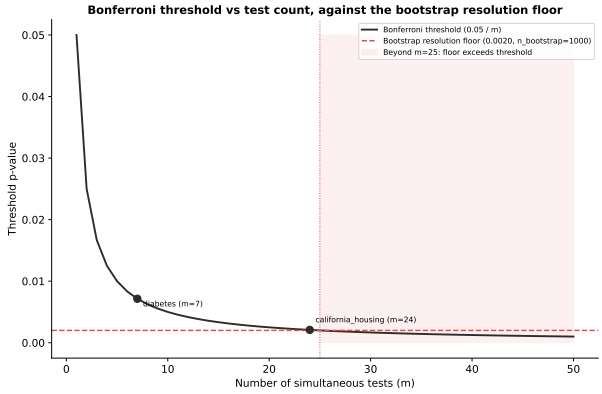


Figure 6: The Bonferroni threshold (descending curve) against the bootstrap test's fixed resolution floor of 0.002 (dashed line). Beyond roughly 25 simultaneous tests, the required threshold drops below the floor, making it mathematically impossible for any bootstrap-tested subgroup to survive correction regardless of effect strength.

its result landed exactly on that floor. Testing even a few additional subgroups in the same run would have pushed the threshold below 0.002, at which point that same subgroup, carrying an identical underlying effect, would have failed to survive correction, not because the effect was weak, but because the test's resolution had run out of room.

To confirm this was a real and fixable limitation, we reran the California Housing audit with the bootstrap resample count increased tenfold, from 1,000 to 10,000. The smallest possible p-value dropped accordingly to 0.0002, and the surviving subgroup's result improved from 0.002 to 0.001, confirming the mechanism and its straightforward remedy.

4.2 Comparing the two test types directly

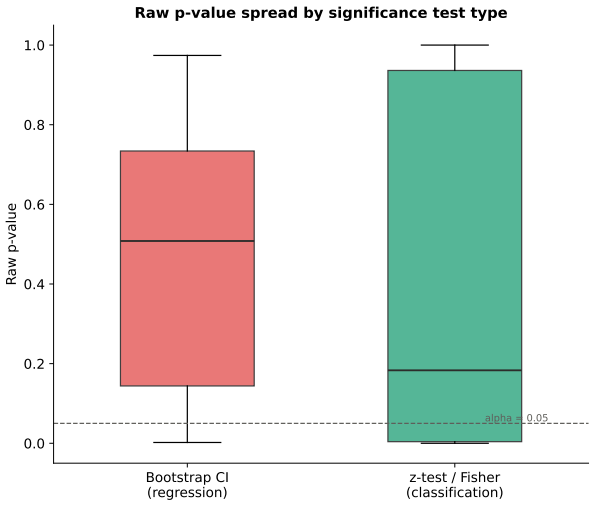


Figure 7: Raw p-value distributions pooled across all four datasets, split by significance test type. Bootstrap-tested results cluster closer to the uncorrected threshold, consistent with their greater sensitivity to correction.

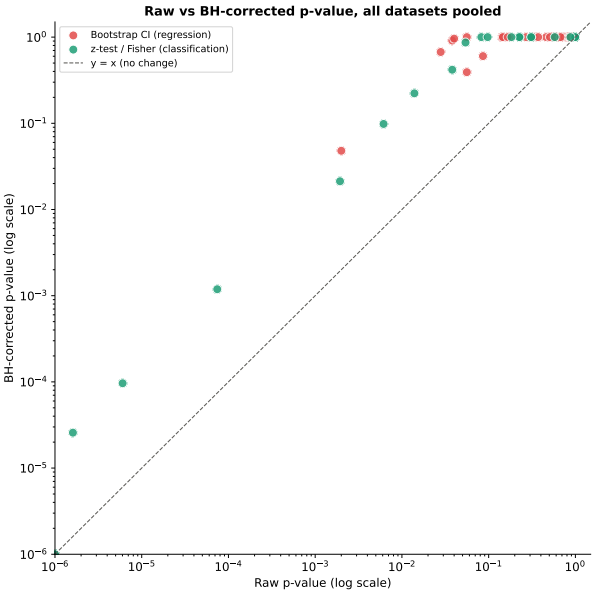


Figure 8: Raw versus Benjamini-Hochberg corrected p-value for every eligible subgroup across all four datasets. Points further above the diagonal were penalized more heavily by correction; bootstrap-tested subgroups sit noticeably further from the diagonal than classification subgroups.

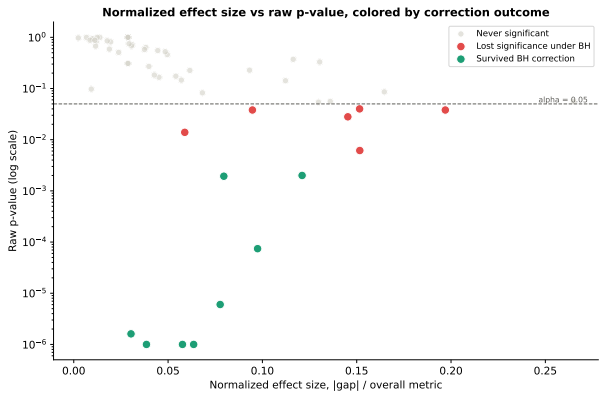


Figure 9: Effect size, expressed as a share of the overall metric so datasets with different scales are comparable, plotted against raw p-value. Subgroups that lost significance under correction tended to sit close to the uncorrected threshold to begin with, indicating borderline rather than overwhelming original results.



Figure 10: Subgroup counts before and after Benjamini-Hochberg correction, shown separately for each dataset.

5. Discussion

The central finding is not that correction reduces the number of significant results, which is true of any correction applied to any dataset by design. The more specific and useful finding is that the reliability of a slice-auditing tool's output depends heavily on which statistical test produced it, a distinction a typical user of the tool would have no obvious reason to investigate. A practitioner auditing a classification model with PySliceKit can reasonably trust that a flagged subgroup will hold up under scrutiny. The same confidence is not warranted for a regression model audited under the tool's default bootstrap settings, where a substantial share of flagged subgroups did not survive even a routine correction check in our experiments.

This has a direct, low-cost remedy. Increasing the number of bootstrap resamples, particularly as the number of subgroups tested in a single run grows, restores resolution and gives genuinely strong effects room to be detected as significant even under strict correction. We suggest that tools relying on resampling-based significance testing scale their resample count to the number of simultaneous tests being run, rather than applying a single fixed default regardless of subgroup count.

It is also worth stating plainly what correction costs. Applying BH or Bonferroni means some subgroups that would previously have been flagged will no longer be. If any of those subgroups represented a real, if moderate, problem, correction means it will be missed. This is the ordinary and well-understood tradeoff of any multiple-comparisons correction: fewer false alarms, at the cost of reduced sensitivity to weaker true effects. We are not arguing that correction should be applied uncritically in every case, only that a tool making significance claims across many subgroups at once should make this tradeoff visible and available to the user, rather than leaving the risk of false positives silently unaddressed.

6. Conclusion

We audited the significance-testing logic inside PySliceKit, a real, open-source tool for identifying hidden subgroup failures in machine learning models. This audit surfaced an actual bug in the tool's bootstrap significance test, which we identified, corrected, and verified. Following the correction, we found that PySliceKit's significance flags are considerably more reliable for classification tasks tested with a z-test than for regression tasks tested with a bootstrap confidence interval, and traced this difference to a hard resolution limit built into resampling-based tests at typical default settings. Every result in this report was independently recalculated by hand, separately from the library's own correction code, and matched in full across all four datasets and both correction methods. We release the raw data, correction reports, and figures behind every number in this report as supplementary material.

References

[1] Benjamini, Y., and Hochberg, Y. (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1), 289-300.

[2] Benjamini, Y., and Yekutieli, D. (2001). The control of the false discovery rate in multiple testing under dependency. *Annals of Statistics*, 29(4), 1165-1188.

[3] Phipson, B., and Smyth, G. K. (2010). Permutation p-values should never be zero: calculating exact p-values when permutations are randomly drawn. *Statistical Applications in Genetics and Molecular Biology*, 9(1).

[4] Storey, J. D. (2002). A direct approach to false discovery rates. *Journal of the Royal Statistical Society: Series B*, 64(3), 479-498.

[5] Dunn, O. J. (1961). Multiple comparisons among means. *Journal of the American Statistical Association*, 56(293), 52-64.

[6] Efron, B. (1979). Bootstrap methods: another look at the jackknife. *Annals of Statistics*, 7(1), 1-26.

[7] Efron, B., and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

[8] Fisher, R. A. (1922). On the interpretation of χ^2 from contingency tables, and the calculation of P. *Journal of the Royal Statistical Society*, 85(1), 87-94.

[9] Wald, A. (1943). Tests of statistical hypotheses concerning several parameters when the number of observations is large. *Transactions of the American Mathematical Society*, 54(3), 426-482.

[10] Chung, Y., Kraska, T., Polyzotis, N., Tae, K. H., and Whang, S. E. (2019). Automated data slicing for model validation: a big data-AI integration approach. *IEEE Transactions on Knowledge and Data Engineering*.

[11] Saleiro, P., Kuester, B., Hinkson, L., London, J., Stevens, A., Anisfeld, A., Rodolfa, K. T., and Ghani, R. (2018). Aequitas: a bias and fairness audit toolkit. *arXiv preprint arXiv:1811.05577*.

[12] Bird, S., Dudík, M., Edgar, R., Horn, B., Lutz, R., Milan, V., Sameki, M., Wallach, H., and Walker, K. (2020). Fairlearn: a toolkit for assessing and improving fairness in AI. Microsoft Technical Report MSR-TR-2020-32.

[13] Bellamy, R. K. E., Dey, K., Hind, M., Hoffman, S. C., Houde, S., Kannan, K., Lohia, P., Martino, J., Mehta, S., Mojsilović, A., et al. (2019). AI Fairness 360: an extensible toolkit for detecting and mitigating algorithmic bias. *IBM Journal of Research and Development*, 63(4/5), 4:1-4:15.

[14] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

[15] Seabold, S., and Perktold, J. (2010). Statsmodels: econometric and statistical modeling in Python. *Proceedings of the 9th Python in Science Conference*, 92-96.

[16] Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261-272.

[17] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362.

[18] McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61.

[19] Hunter, J. D. (2007). A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95.

[20] Pace, R. K., and Barry, R. (1997). Sparse spatial autoregressions. *Statistics & Probability Letters*, 33(3), 291-297.

[21] Wolberg, W. N., and Mangasarian, O. L. (1990). Multisurface method of pattern separation for medical diagnosis applied to breast cytology. *Proceedings of the National Academy of Sciences*, 87(23), 9193-9196.

[22] Street, W. N., Wolberg, W. H., and Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. *Proceedings of SPIE: Biomedical Image Processing and Biomedical Visualization*, 1905, 861-870.

[23] Kohavi, R. (1996). Scaling up the accuracy of naive-Bayes classifiers: a decision-tree hybrid. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, 202-207.

[24] Efron, B., Hastie, T., Johnstone, I., and Tibshirani, R. (2004). Least angle regression. *Annals of Statistics*, 32(2), 407-499.

[25] Dua, D., and Graff, C. (2019). UCI Machine Learning Repository. University of California, Irvine, School of Information and Computer Sciences. Available at: `http://archive.ics.uci.edu/ml`.

[26] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.

[27] Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B*, 20(2), 215-242.

[28] Hoerl, A. E., and Kennard, R. W. (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55-67.

[29] Sokolova, M., and Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427-437.

[30] Willmott, C. J., and Matsuura, K. (2005). Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Climate Research*, 30(1), 79-82.

Appendices

Appendix A. Bug fix detail. The original bootstrap significance test computed its reported p-value as a one-sided percentile position (the fraction of resampled values at least as extreme as the overall metric in one direction), while the significance decision itself was based on a two-sided check (whether the overall metric fell outside the middle 95% of the resampled distribution). This mismatch caused subgroups with strong effects in one direction to report correct, small p-values, while subgroups with equally strong effects in the other direction reported p-values close to 1. The fix computes the p-value as two times the smaller of the two one-sided fractions, the standard construction for a two-tailed resampling p-value.

Appendix B. Independent verification procedure. For each dataset, the Benjamini-Hochberg procedure was reimplemented as follows: sort all eligible raw p-values ascending; for each rank i (starting at 1) out of m total tests, compute the critical value $(i/m) \times 0.05$; find the largest rank at which the raw p-value is less than or equal to its critical value; all subgroups at or below that rank are significant. Bonferroni was verified by comparing each raw p-value directly against $0.05/m$. Both procedures were implemented using only base Python and numpy, without any statistics library, and compared line by line against the library's own `apply_correction()` output.

Supplementary Material

The following are included alongside this report:

- Raw, uncorrected results for each dataset (`01_raw_results.csv`)
- Raw p-value listings per dataset (`02_raw_pvalues.txt`)
- Independent verification comparison files, showing agreement between hand-calculated and library-calculated correction outcomes for every subgroup (`03_verification_comparison.txt`)
- Environment and package version capture, for exact reproducibility (`00_environment.txt`)
- Consolidated master report with all summary statistics (`00_MASTER_REPORT.txt`)
- Pooled segment table across all four datasets, used to generate the combined figures (`pooled_segment_table.csv`)
- All figures referenced above, at print resolution

These files are sufficient to reproduce every number reported in this paper without rerunning the underlying experiments.